

TIPOS DE DATOS ABSTRACTOS (TADs)

En el presente proyecto se implementaron Tipos Abstractos de Datos (TAD) dinámicos propios, sin el uso de estructuras de C++, con el objetivo de reforzar la comprensión de estructuras enlazadas, manejo de memoria dinámica y diseño modular.

A continuación se describen las estructuras implementadas.

1) **Nodo<T>**

Representa la unidad básica de almacenamiento utilizada en las estructuras dinámicas del sistema. Es una estructura genérica que permite almacenar cualquier tipo de dato mediante el uso de plantillas (template).

Estructura Interna

Cada nodo contiene:

- T dato → información almacenada.
- Nodo<T>* siguiente → puntero al siguiente nodo en la estructura.

Propósito en el sistema

El nodo sirve como base estructural para:

- ListaSimple<T>
- ListaCircular<T>
- Stack<T>

Permite la construcción de estructuras dinámicas enlazadas sin necesidad de arreglos estáticos.

2) **ListaSimple<T>**

es un Tipo Abstracto de Datos que implementa una lista simplemente enlazada genérica. Los elementos se organizan secuencialmente mediante nodos conectados por punteros.

Estructura Interna

- Nodo<T>* cabeza → apunta al primer nodo de la lista.
- Cada nodo apunta al siguiente mediante siguiente.
- El último nodo apunta a nullptr.

Operaciones Principales

- insertarFinal(T dato) → Inserta un elemento al final.
- eliminar(T dato) → Elimina el primer nodo que contenga el dato.
- buscar(T dato) → Verifica si un elemento existe.
- estaVacia() → Indica si la lista está vacía.
- obtenerPrimero() → Devuelve el primer elemento sin eliminarlo.
- extraerPrimero() → Extrae y elimina el primer elemento.
- Destructor → Libera memoria dinámicamente.

Uso en el Proyecto

Se utiliza para:

- Representar la mano de cada jugador.
- Implementar la bolsa temporal utilizada para barajar cartas.
- Almacenar los castigos personalizados.

Esta estructura permite inserciones y eliminaciones dinámicas durante la ejecución del juego.

3) ListaCircular<T>

es una lista enlazada circular simple donde el último nodo apunta nuevamente al primero, formando un ciclo continuo.

Estructura Interna

- Nodo<T>* cabeza
- Nodo<T>* cola
- cola->siguiente apunta a cabeza.

Operaciones Principales

- insertar(T dato) → Inserta un elemento al final manteniendo la circularidad.
- getCabeza() → Devuelve el nodo inicial.
- estaVacia() → Verifica si la lista está vacía.
- buscarAnterior(Nodo<T>* actual) → Devuelve el nodo anterior al actual.

- Destruir → Libera memoria correctamente.

Uso en el Proyecto

Se utiliza para:

- Almacenar los jugadores.
- Gestionar el orden de turnos.
- Permitir cambio de dirección mediante la carta Reverse.

La naturaleza circular facilita la rotación continua de turnos sin necesidad de reiniciar recorridos manualmente.

4) Stack<T> (Pila)

Definición

El Stack<T> es un Tipo Abstracto de Datos que implementa una pila bajo el principio LIFO (Last In, First Out). El último elemento insertado es el primero en salir.

Estructura Interna

- Nodo<T>* tope → apunta al nodo superior de la pila.
- Cada nuevo elemento insertado se convierte en el nuevo tope.

Operaciones Principales

- push(T dato) → Inserta un elemento en la cima.
- pop() → Extrae y devuelve el elemento superior.
- peek() → Devuelve el elemento superior sin eliminarlo.
- estaVacia() → Indica si la pila está vacía.
- Destruir → Libera memoria dinámicamente.

Uso en el Proyecto

Se utiliza para:

- Representar el mazo principal.
- Representar el mazo de descarte.

Su comportamiento LIFO modela correctamente la lógica del juego, donde las cartas se roban desde la parte superior del mazo